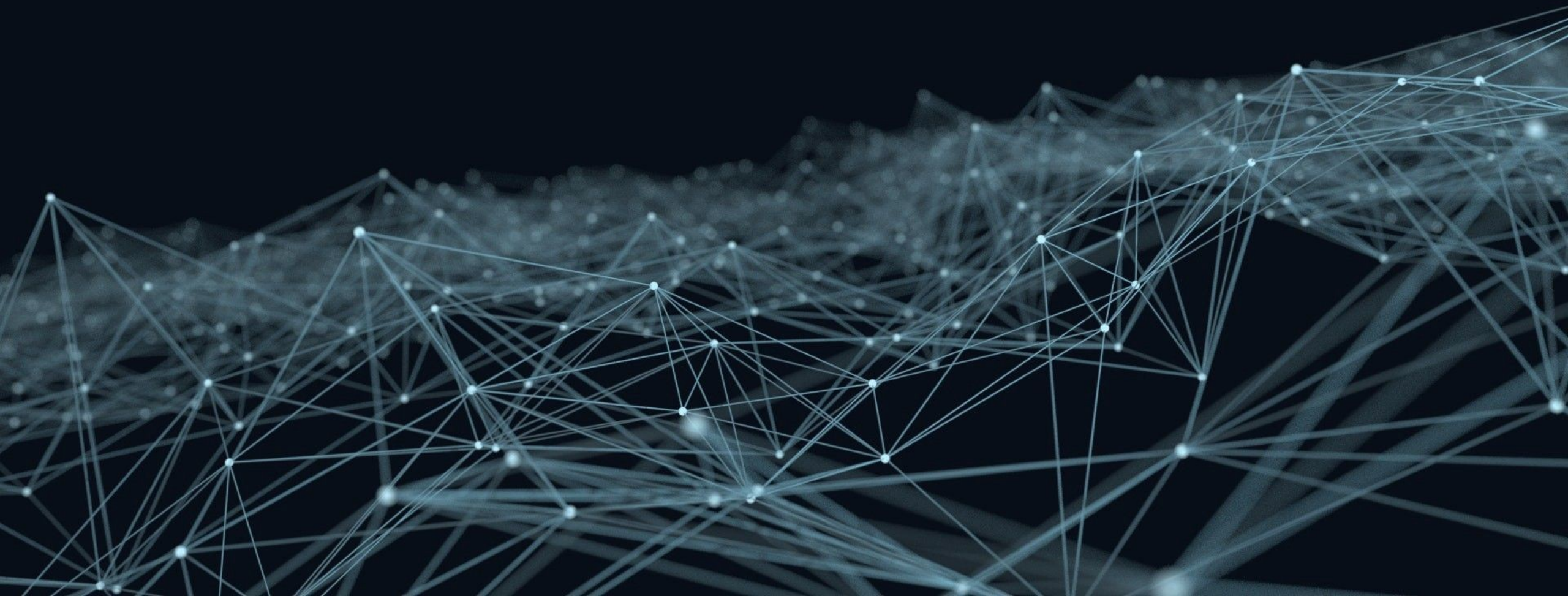
Java Full Stack Development

1. IntelliJ Setup and Navigation



Module 0

- The sections on IntelliJ and Copilot are “ramp up” sections
  - You will be expected to use both IntelliJ and Copilot throughout the course
- The material covered in class is intended as a broad and fast introduction
  - The slides will go into more depth that we will during the lectures
  - The additional materials are for your own use during the rest of the course
- There are additional resources in the “Resources” folder for this module
  - These are intended for self study
  - Take advantage of them if you are new to either IntelliJ or Copilot
- The labs will walk you through basic usage

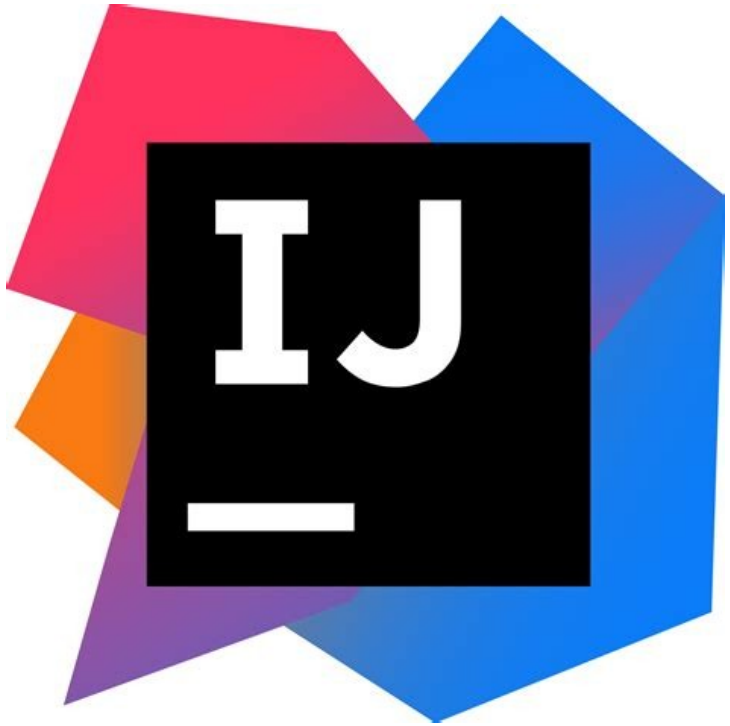
Module Topics

- Introduction to IntelliJ
- Project setup and configuration
- Code Inspections and intentions
- Refactoring
- Navigation in a large codebase

IntelliJ Configuration

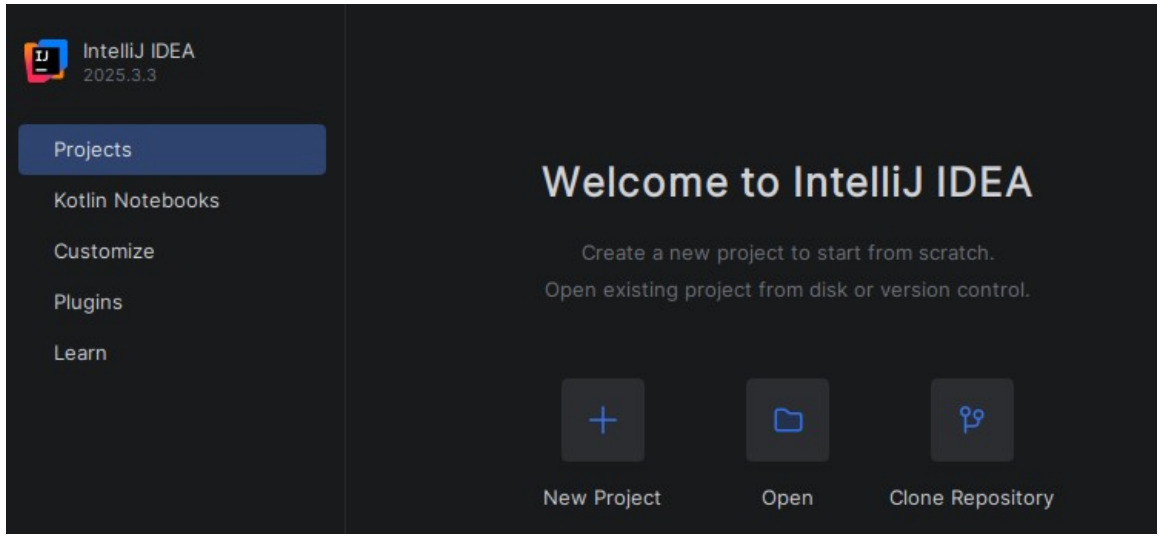
IntelliJ

- IntelliJ IDEA is a polyglot IDE developed by JetBrains
  - Java native support is built into the core engine
  - IntelliJ tools operate on a fully resolved semantic model of the code, not a best-effort parse
    - For example: static analysis tools, refactoring engine, code generation, and debugger
    - Every inspection, 'Extract Method', 'Find Usages' execution operates against the real type graph of a project
  - In contrast, VS Code uses language server extensions
    - Provides language server protocol (LSP) support
    - Architecturally more limited
    - These imitations impact large-scale refactoring, Spring-specific inspection, and deep framework integration



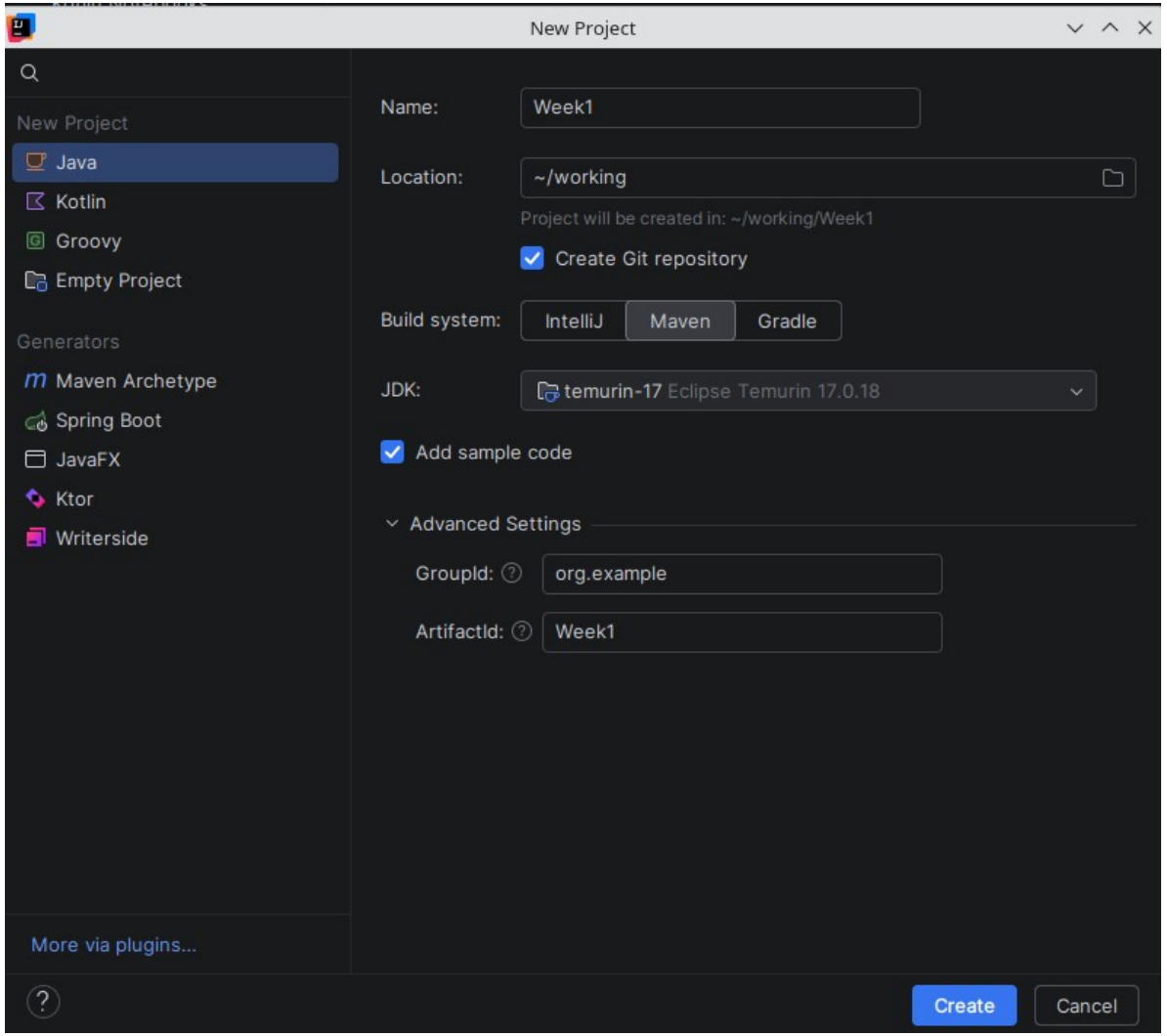
Project Setup

- IntelliJ IDEA organizes work on a project basis
- Setting are set up at the IDE level
  - These are defaults for each project
  - These can be overridden for project specific settings



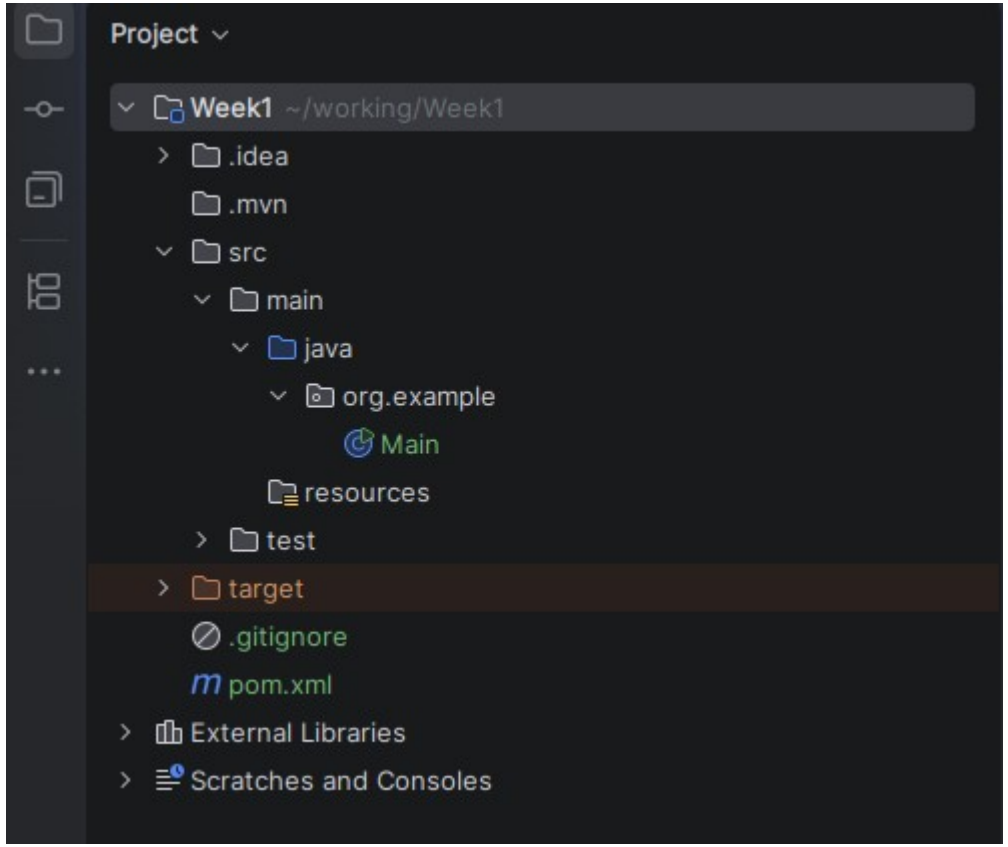
Project Setup

- There are two JVMs used
  - IntelliJ is a Java application so the system JVM is used, in this case Java 25 to run the IDE
  - The project JVM is used to compile and run the code in the project which may be any other Java version
  - In the screenshot, the project JVM is Java 17
- The Maven build tool is selected



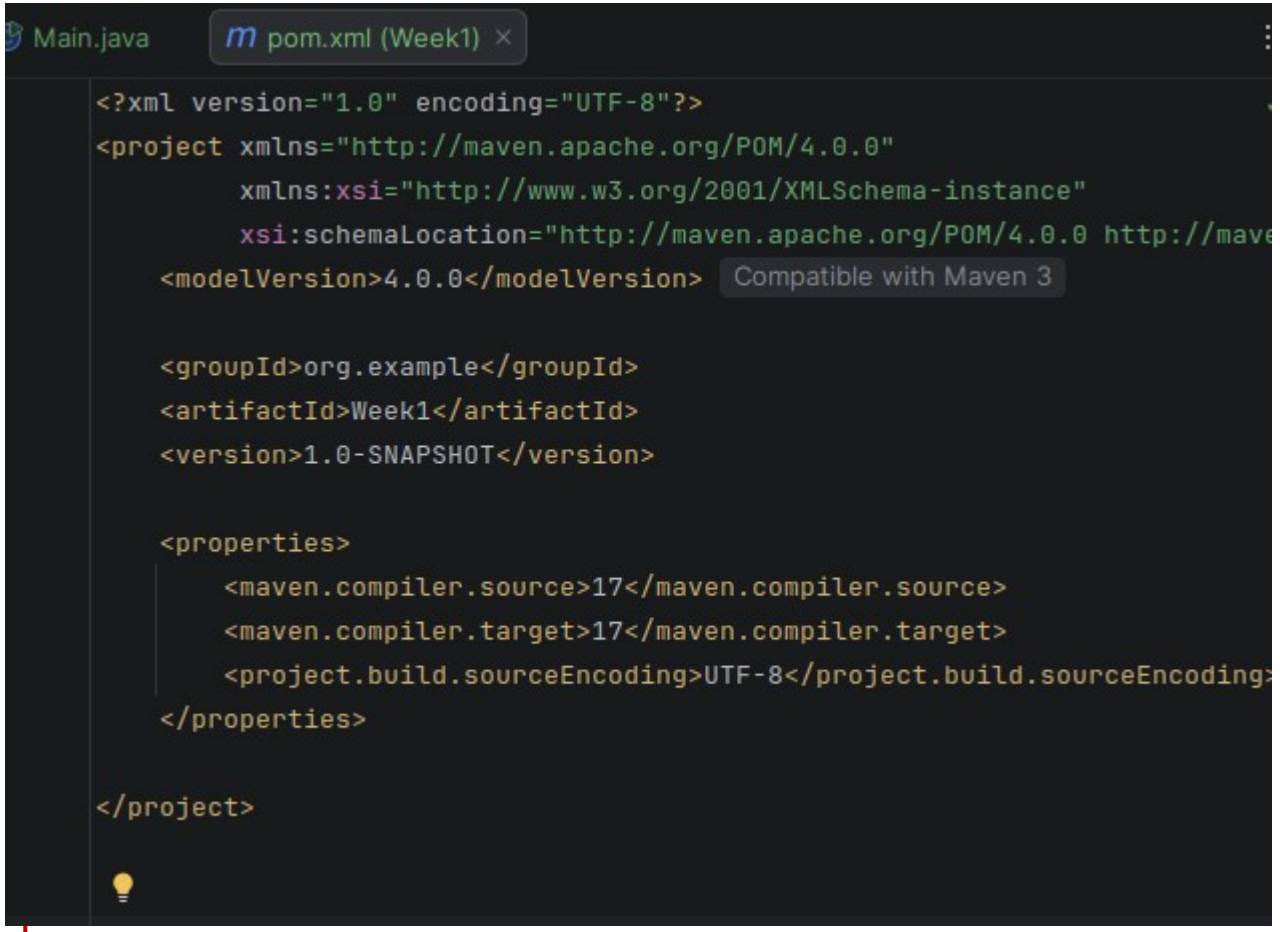
Project Setup

- The project shown was generated by the sample code option
  - The directory structure is a standard Maven project structure
  - The src folder contains three subfolders
    - The main folder that contains the code
    - The test folder that contains the unit tests
    - The resources folder that contains other project artifacts like config files, images, etc
  - The compiled and packaged code is in the target directory
  - The External Libraries are all of the class libraries used in the project



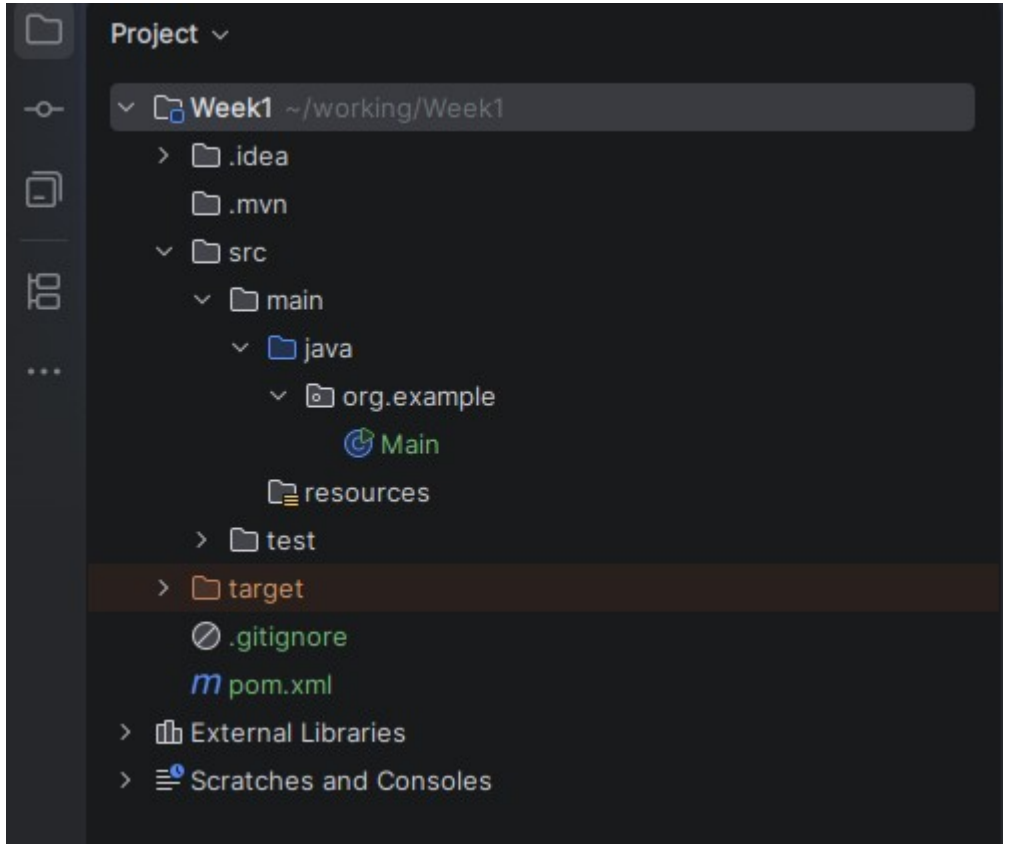
Pom.xml File

- The pom.xml file is the single source of truth for the project
  - Note that pins the Java version to 17
  - And pins the file encoding to UTF-8



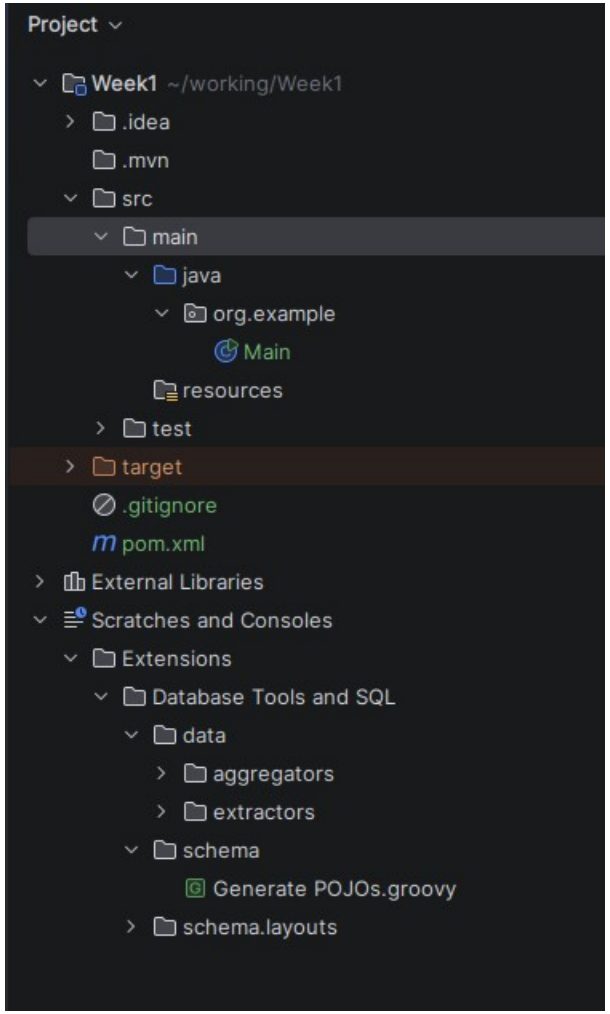
Scratches

- The Scratches folder holds temporary, project-independent files that are outside the source tree
- A scratch file is a throwaway editor buffer that supports full language features
  - Scratch files are not part of any project
  - Stored globally in your IntelliJ config directory
  - The scratch file is visible in other projects



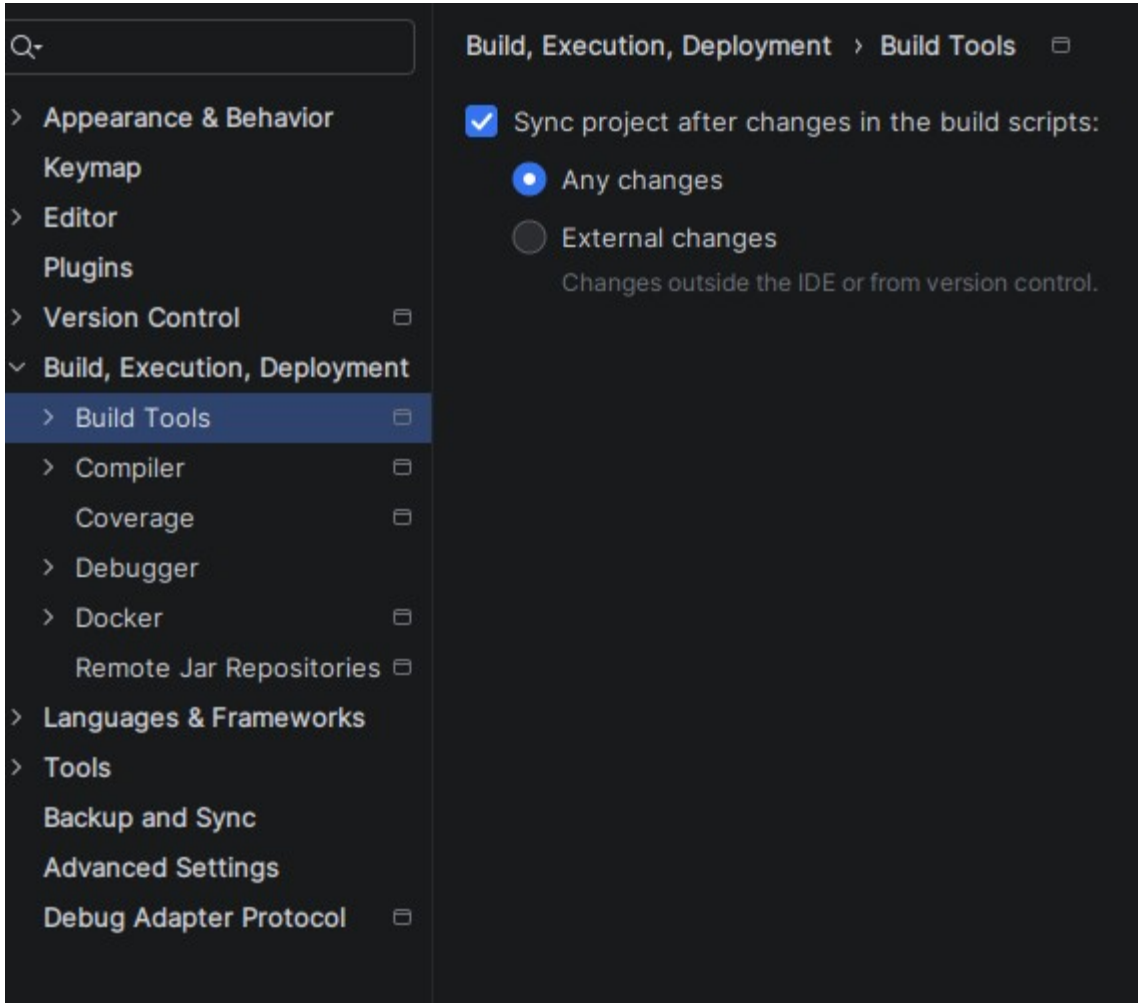
Consoles

- The Consoles are saved interactive console sessions
- There are three types available
  - Database consoles
    - An .sql file that IntelliJ saves so queries are not lost when the IDE is closed
    - We will be using this console extensively in week 3
  - HTTP client consoles
    - Unsaved .http request files land here before they are explicitly saved to the project
    - We will be using this console extensively throughout the course
  - Language consoles
    - When a Groovy or other REPL-style console is opened, the session is stored here



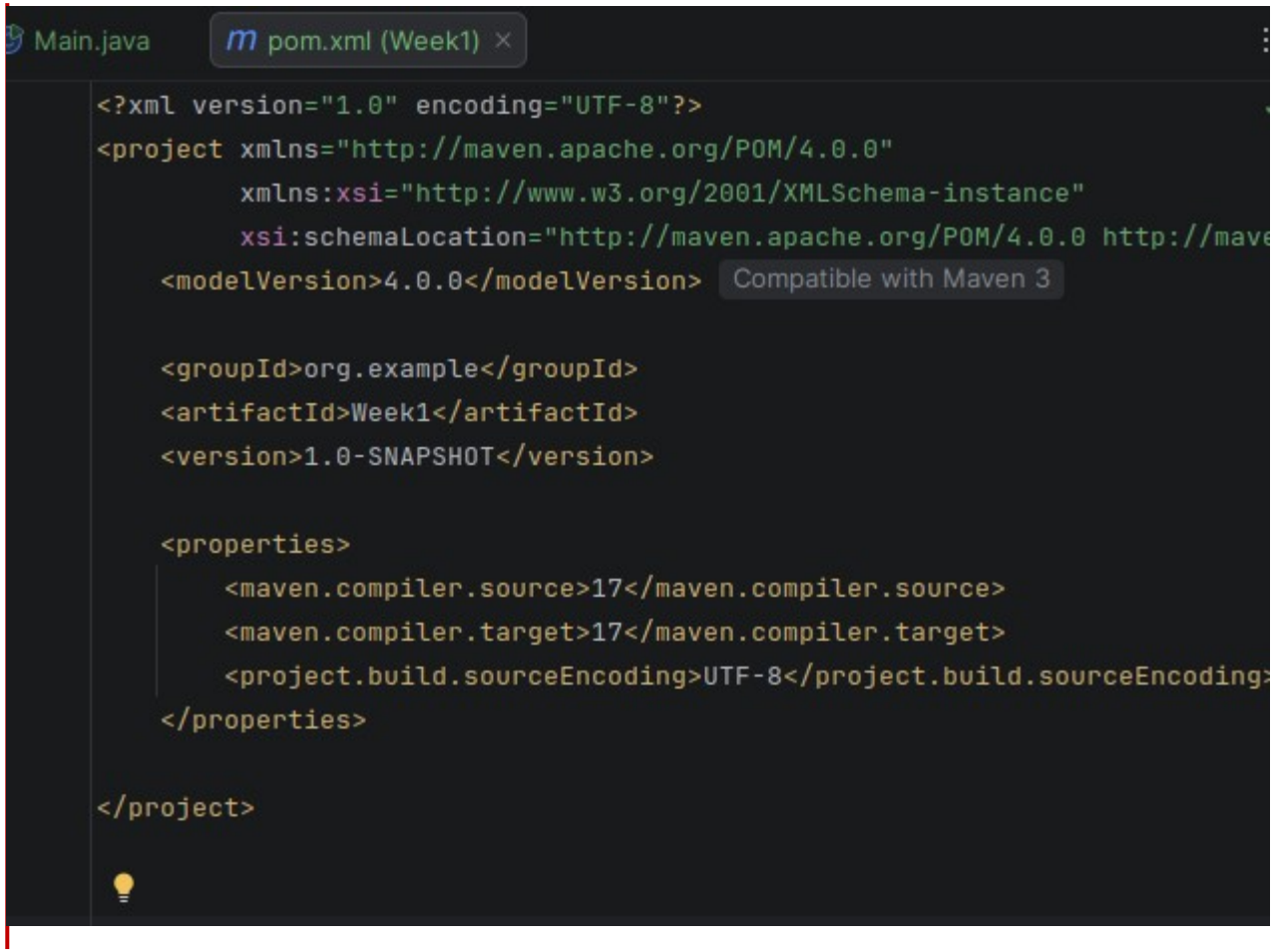
Auto-Import

- Maven auto-import
  - IntelliJ can watch the pom.xml file
    - Can automatically synchronize its internal project model whenever a dependency is modified
    - Without this, manually triggering Reload Project every the pom.xml is edited
  - This has to be turned on in the settings as shown here
  - Triggered every time a Spring Boot starter, a dependency added, or change is made to a a plugin version
  - Without auto-import phantom compilation errors can occur that are simply stale classpath entries
  - There is an upcoming module on Maven



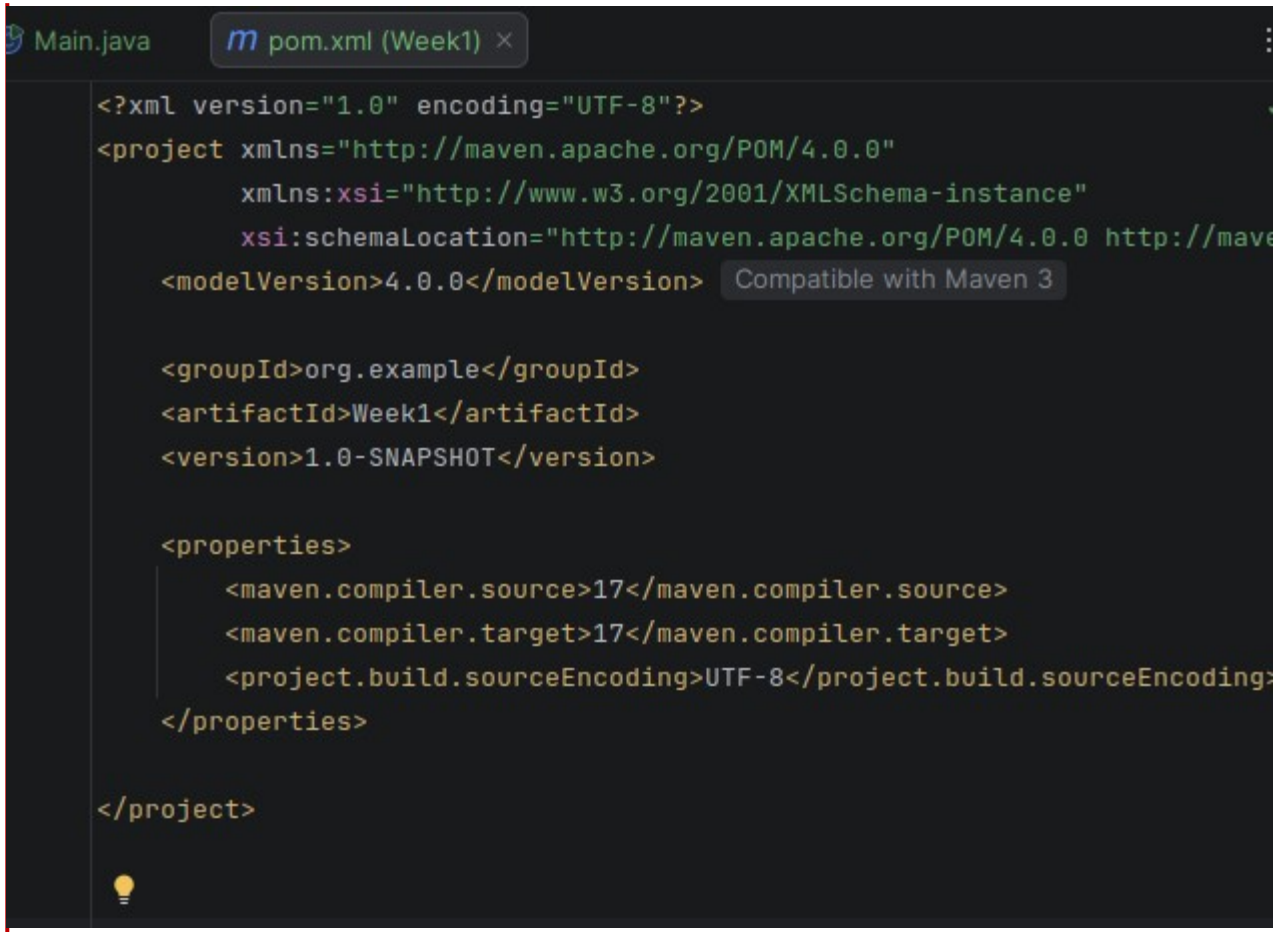
File Encoding

- File encoding
  - On Windows development machines, the default system encoding is often Windows-1252 or CP-1252
  - Java expects UTF-8 for source files, resource files, and configuration
  - Mismatched encoding causes silent data corruption in string literals
    - Particularly visible in JSON payloads, SQL statements with special characters, and internationalized error messages
    - Best case: compile error
    - Worst case: causes source files to be misread as garbled tokens



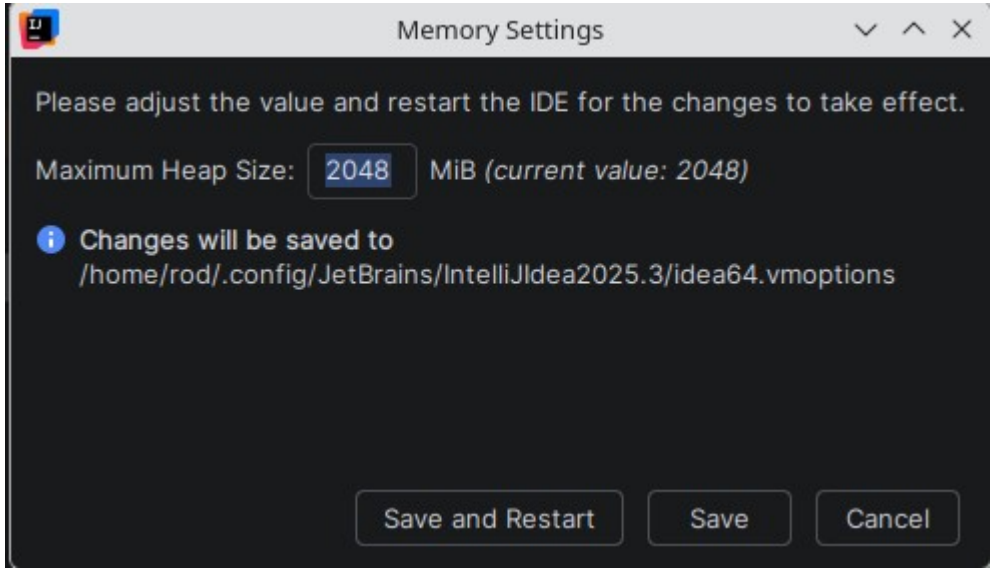
File Encoding

- How corruption typically happens
  - Created or edited on Windows in Notepad
    - This app historically defaulted to UTF-16
  - Copied from a Windows SharePoint or Teams file share onto a Linux build server
  - Generated by a tool (Word, Excel export) that produces UTF-16 by default
  - Edited by a developer whose IDE was incorrectly configured



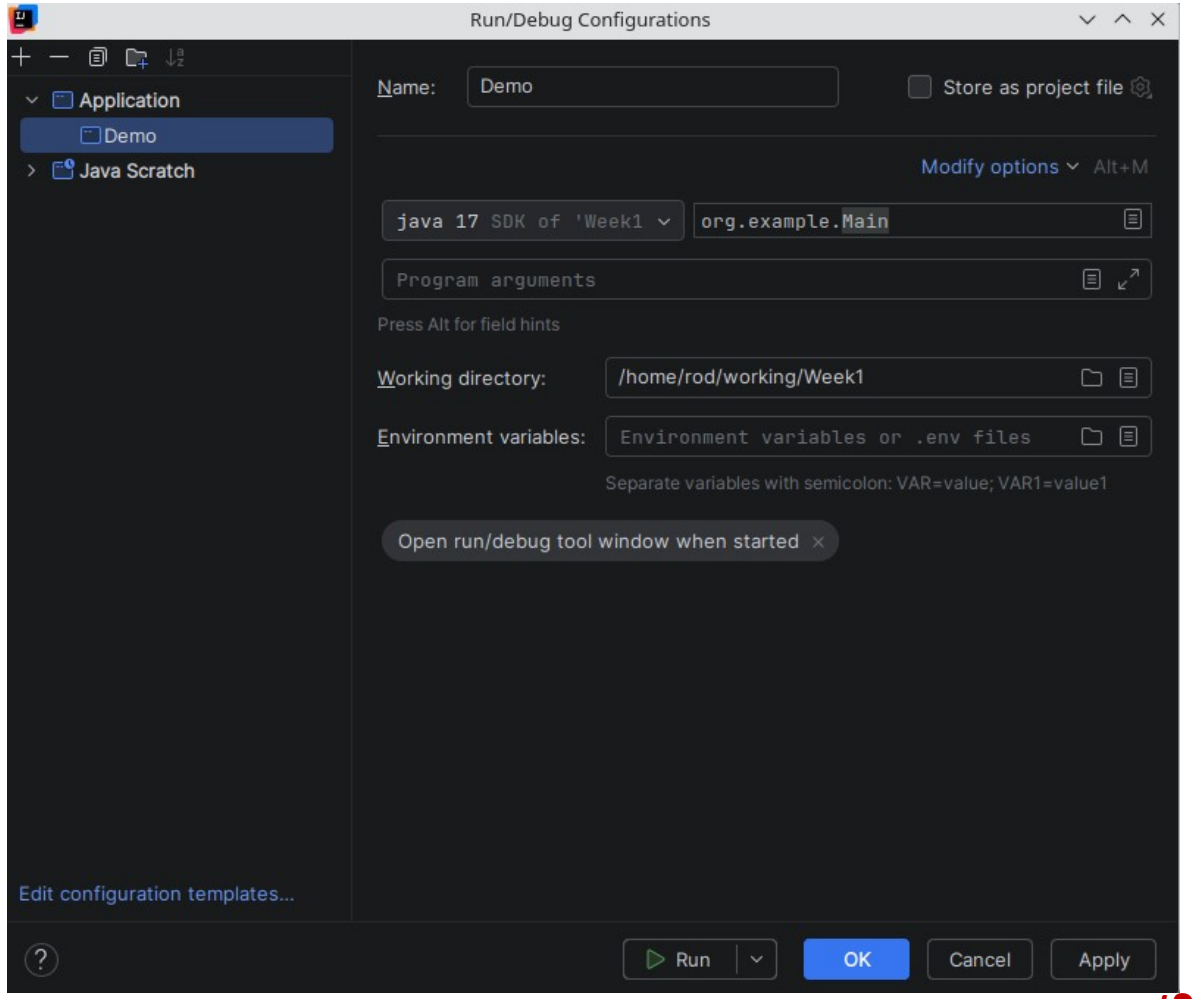
IDE Memory

- IDE memory settings
  - Found under the Help menu
  - Spring Boot projects with multiple modules, Kafka, Oracle drivers, and test containers can require substantial heap
  - IntelliJ ships with conservative defaults (512MB–1GB) that cause noticeable slowdown on large projects
    - Refers to the memory used by the IDE, not the JVM used by the application
  - Increasing the heap eliminates the most common source of IDE lag on Spring builds
  - Set minimum to 2048 MB for any project with multiple Spring Boot modules
    - 4096 MB is reasonable on a 16GB+ machine



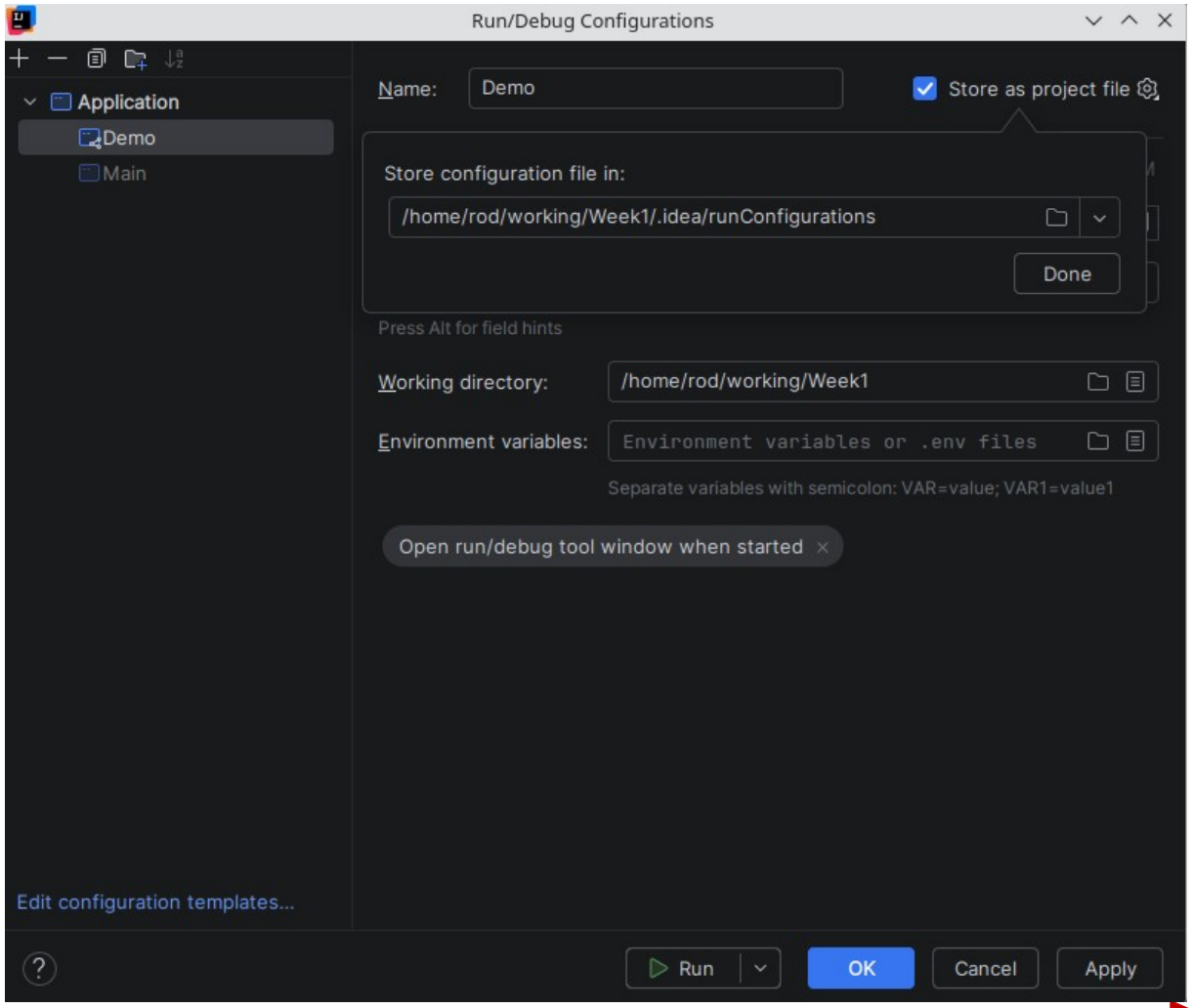
Run Configurations

- A saved, named set of instructions that tells IntelliJ how to launch something
  - Can be a Java application, a test class, a Maven goal, a Docker container, a remote debugger connection or a Spring Boot applications.
  - Determines exactly what happens when the run button is pressed
  - The first time the the green play button is selected a temporary Run Configuration is created but not saved



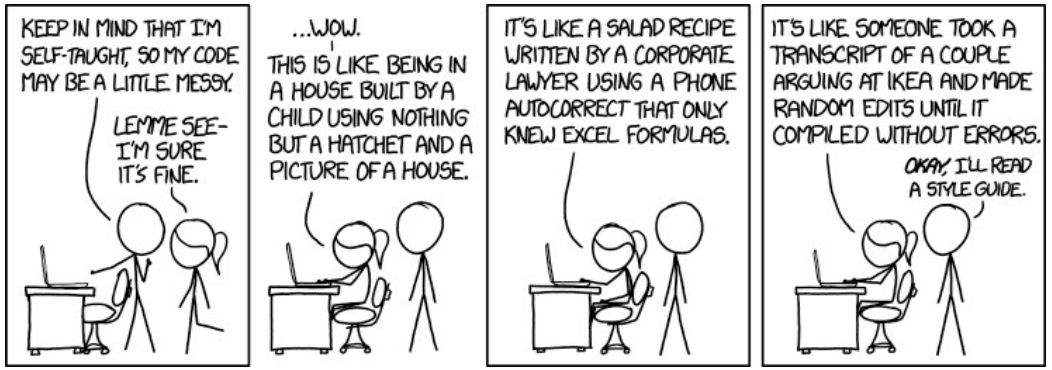
Run Configurations

- Three states for a Run Configuration
  - Temporary
    - Created automatically when you click Run next to a main() method
    - IntelliJ keeps only the last few and then deletes old ones
  - Permanent
    - Saved locally in your IDE
    - Survives IDE restarts
    - Created by saving a temporary configuration or by creating one manually
  - Shared
    - Stored .idea/runConfigurations/ as an XML file and committed to Git
    - Every developer who clones the repo gets the same Run Configurations automatically



Code Style and Editor Config

- The merge conflict problem
  - Two developers working on the same file
  - One is configured to use 4-space indents
  - The other is configured to use tabs
  - Neither changes the code logic
  - IntelliJ auto-formats on save so every single line in the file appears as a change in Git
    - Git sees this as every line being deleted and rewritten
  - The goal of code style enforcement is to make every line in every diff meaningful




```
- public Employee findById(Long id) {
-     return repository.findById(id)
-         .orElseThrow(() -> new EmployeeNotFoundException(id));
- }
+ public Employee findById(Long id) {
+   return repository.findById(id)
+       .orElseThrow(() -> new EmployeeNotFoundException(id));
+ }
```


Image Credit: https://xkcd.com/1513

Code Style and Editor Config

- Code style covers two distinct categories of rules
- Formatting rules
  - Purely cosmetic and do not affect compilation or runtime behaviour
  - They only affect how the source text looks on screen and on disk
    - Indentation (tabs vs spaces, how many spaces)
    - Line endings (LF vs CRLF)
    - Where braces go (same line or next line)
    - Maximum line length
  - Controlled by .editorconfig and IntelliJ's code style settings

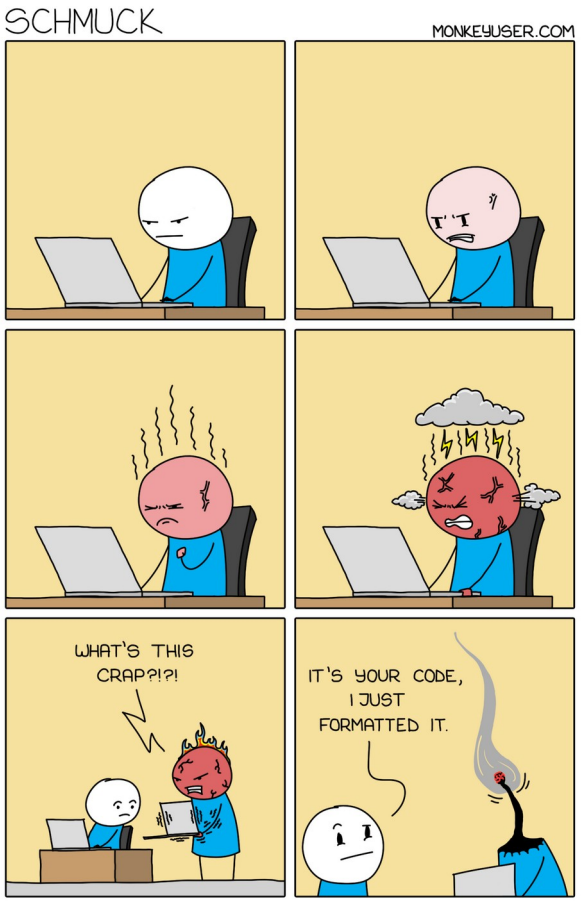


Image Credit: https://github.com/ichimax/Java-Style-Guide

Code Style and Editor Config

- Code quality rules
  - These affect the structure and correctness of the code itself
    - Naming conventions (camelCase for methods, PascalCase for classes)
    - Whether to use `var` or explicit types
    - Whether fields should be `final`
    - Import organization (no wildcards, grouped by package)
    - Complexity limits like max method length
  - Controlled by IntelliJ inspections, Checkstyle, or SonarQube

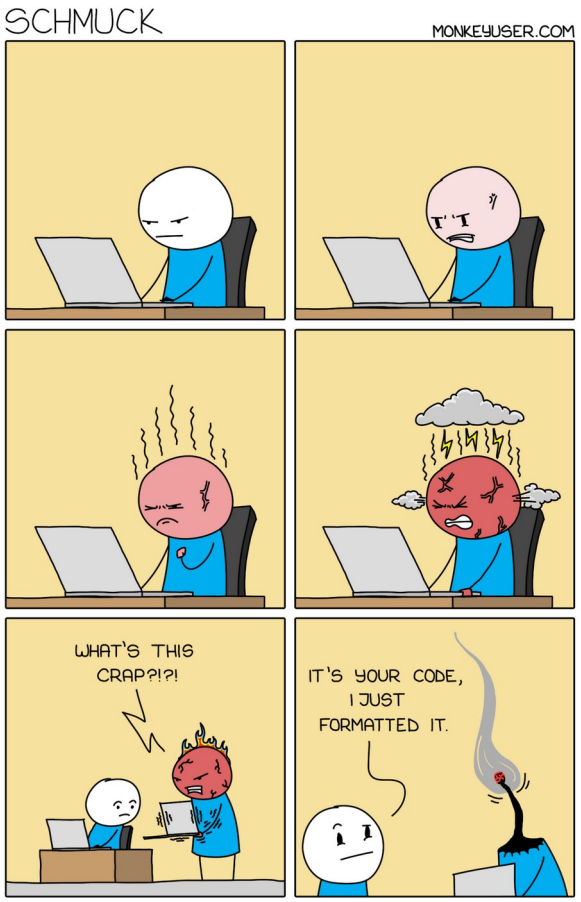
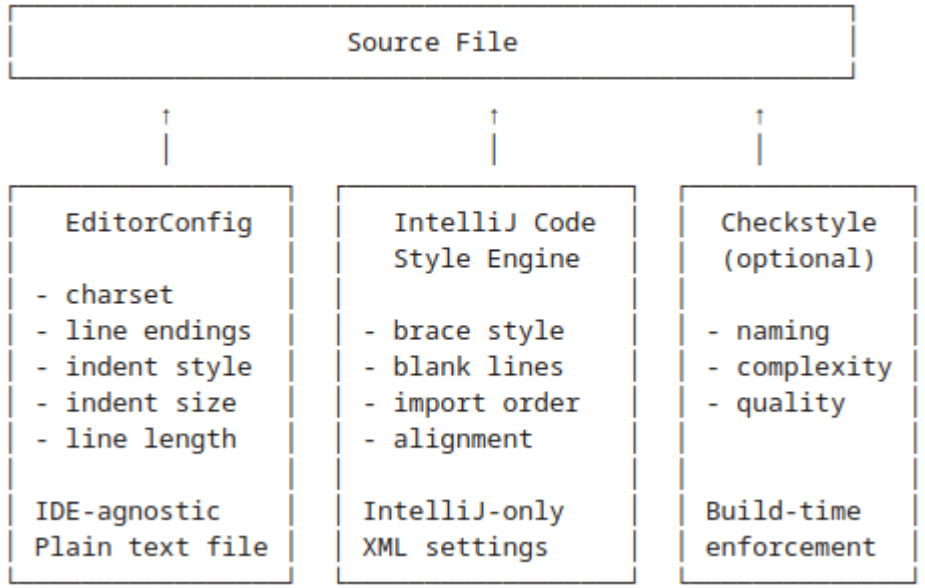


Image Credit: https://github.com/ichimax/Java-Style-Guide

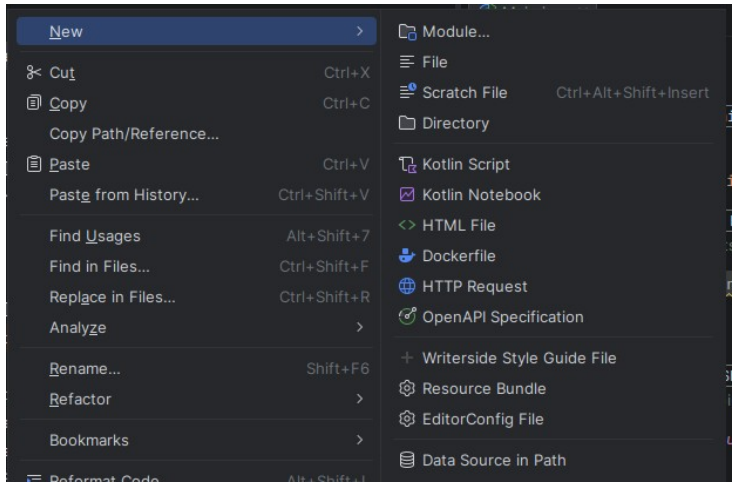
Code Style and Editor Config

- Understanding the tool stack
  - The three configuration tools that make up the stack operate at different levels
  - The EditorConfig applies across the IDE to all files regardless of what the file contains
  - The Code Style Engine is a syntactic checker that ensures the syntax of the code is correct
    - Language specific, different rules for Java versus yaml, for example
  - Checkstyle can be thought of as a semantic checker that looks at what the code "means"



Editor Config

- EditorConfig is the baseline
  - Plain text file ini format that every modern editor understands
  - It enforces the most fundamental formatting rules in a way that works regardless of which IDE each developer uses



ini

```ini
# Root marker — tells editors to stop looking for
# .editorconfig files in parent directories
root = true

# Rules that apply to ALL files
[*]
charset = utf-8
end_of_line = lf
indent_style = space
indent_size = 4
trim_trailing_whitespace = true
insert_final_newline = true

# Override for Java files specifically
[*.java]
indent_size = 4
max_line_length = 120
```

EditorConfig

- When you open a file, the editor:
  - Looks for a .editorconfig file in the same directory as the file
  - If not found, walks up the directory tree looking for one
  - Stops walking when it finds root = true
  - Applies the matching rules to the open file
- This hierarchical lookup means
  - Use a root .editorconfig for general rules
  - Override specific rules in sub-directories

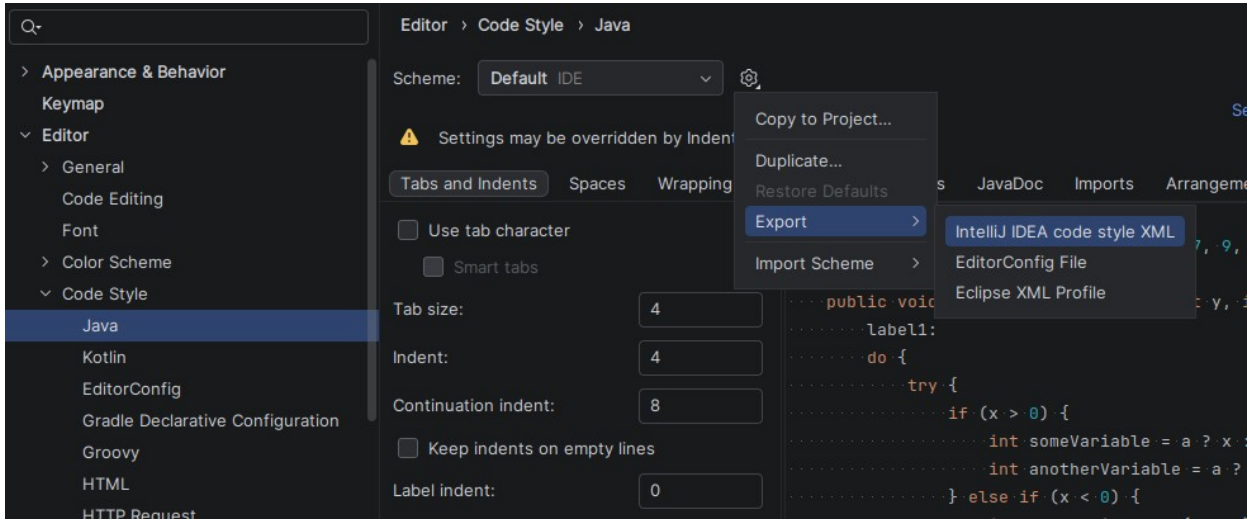
```ini
# Root marker — tells editors to stop looking for
# .editorconfig files in parent directories
root = true

# Rules that apply to ALL files
[*]
charset = utf-8
end_of_line = lf
indent_style = space
indent_size = 4
trim_trailing_whitespace = true
insert_final_newline = true

# Override for Java files specifically
[*.java]
indent_size = 4
max_line_length = 120
```

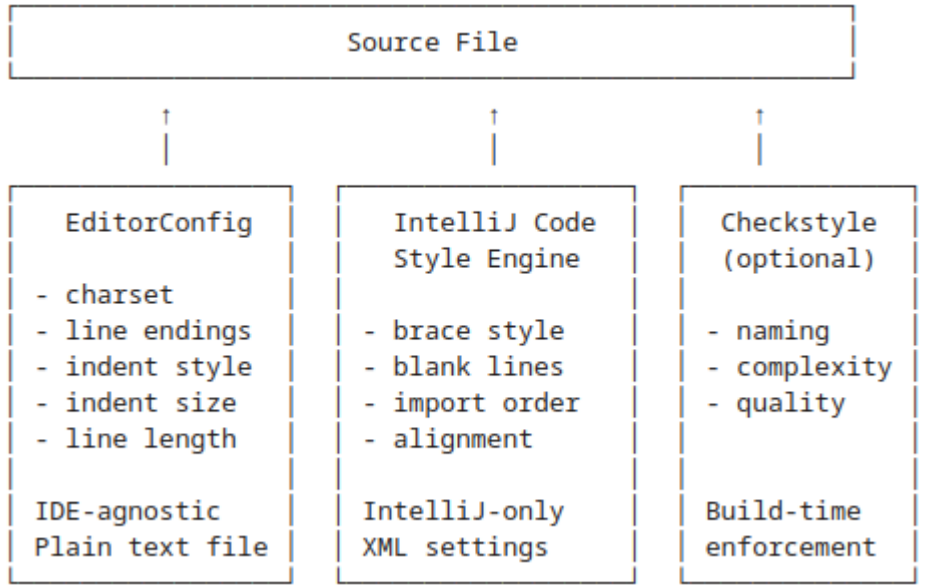
Code Style Engine

- Applies IntelliJ-specific settings that EditorConfig cannot express
  - Includes format items like
    - Brace placement
    - Import organization
    - Method chaining alignment
- Stored in IntelliJ's code style XML
  - Can be exported and committed to the repository
  - Ensures every team member's IntelliJ uses identical settings



Code Style Engine

- Relationship between the two tools
  - EditorConfig
    - Baseline rules for ALL editors
    - For consistency across environments
    - VS Code, Vim, Emacs, IntelliJ
  - IntelliJ Code Style
    - Richer rules for IntelliJ specifically
    - Respects EditorConfig as the foundation
    - Adds language-specific formatting on top
    - When a rule exists in both, EditorConfig takes precedence



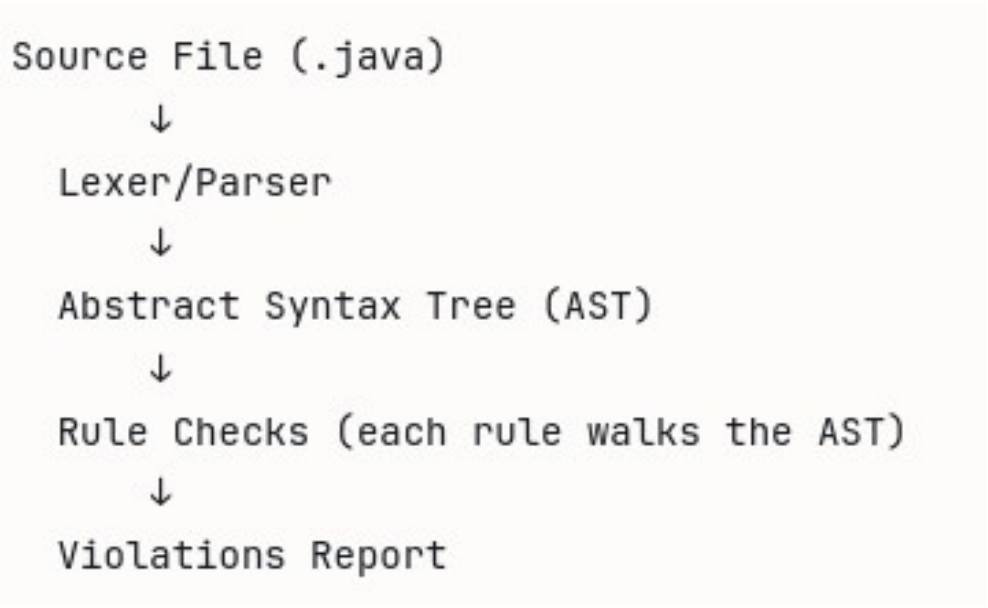
Checkstyle

- Checkstyle is a static analysis tool
  - Automatically checks Java source code against a defined set of coding rules
  - Reads .java source files as text and as ASTs and verifies that they conform to the defined rule-set
    - Does not compile the code
    - Does not run it
    - Does not analyze runtime behavior
    - Purely analyses the structure and style of the source text
  - Checkstyle is the enforcement layer
    - Code does not build until the violation is fixed

```
EditorConfig          →  Your editor fixes formatting as you type
IntelliJ Inspections  →  The IDE warns you in real time
Checkstyle            →  The BUILD FAILS if rules are violated
```

Checkstyle

- Ships with standard pre-built configurations
  - These are used as a starting point for developing a customized style guide
- Google Java style guide
  - Used by Google internally and published as an open standard
  - Most widely adopted in modern Java teams
- Sun coding conventions
  - Original Java coding standard from Sun
  - Included mainly for backward and historical compatibility



Checkstyle

- Integrating Checkstyle into Maven
  - Checkstyle is added to the pom file as a plugin as shown
  - Can be run automatically as part of the build process
  - Or run manually

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-checkstyle-plugin</artifactId>
            <version>3.3.1</version>
            <configuration>
                <!-- Path to your ruleset file -->
                <configLocation>config/checkstyle/checkstyle.xml</configLocation>
                <!-- Path to suppression file (rules to ignore) -->
                <suppressionsLocation>
                    config/checkstyle/suppressions.xml
                </suppressionsLocation>
                <!-- Fail the build on violation -->
                <failsOnError>true</failsOnError>
                <!-- Also fail on warnings -->
                <violationSeverity>warning</violationSeverity>
                <!-- Include test sources -->
                <includeTestSourceDirectory>true</includeTestSourceDirectory>
            </configuration>
            <executions>
                <execution>
                    <id>checkstyle-validation</id>
                    <!-- Binds to verify phase — runs before package -->
                    <phase>verify</phase>
                    <goals>
                        <goal>check</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

Checkstyle

- Checkstyle configuration file
  - The configuration file defines exactly which rules are active and how they are configured
  - The example of the left is a snippet of part of a Checkstyle configuration file

```
<!-- ═══════════════════════════════════════ -->
<!-- CODE QUALITY                            -->
<!-- ═══════════════════════════════════════ -->

<!-- No empty catch blocks -->
<module name="EmptyCatchBlock"/>

<!-- No empty blocks in general -->
<module name="EmptyBlock"/>

<!-- Always use braces, even for single-line if/for/while -->
<module name="NeedBraces"/>

<!-- No magic numbers (except -1, 0, 1, 2) -->
<module name="MagicNumber">
    <property name="ignoreNumbers" value="-1, 0, 1, 2"/>
</module>

<!-- Equals and hashCode must both be overridden or neither -->
<module name="EqualsHashCode"/>

<!-- No System.out.println in production code -->
<module name="Regexp">
    <property name="format" value="System\.out\.println"/>
    <property name="illegalPattern" value="true"/>
    <property name="message"
              value="Use a logger instead of System.out.println"/>
</module>
```

Checkstyle

- Checkstyle configuration file
  - The location of the file is specified in the pom file as shown
  - However, putting it in config/checkstyle/ location is the de facto standard for enterprise Java projects.

```
<configuration>
    <configLocation>config/checkstyle/checkstyle.xml</configLocation>
    <suppressionsLocation>config/checkstyle/suppressions.xml</suppressionsLocation>
</configuration>
```

```
project-root/
├── config/
│   └── checkstyle/
│       ├── checkstyle.xml
│       └── suppressions.xml
├── pom.xml
└── src/
```

Checkstyle

- Cyclomatic complexity
  - One of the most useful rules
  - Directly correlates with code quality, testability, and maintainability
  - Counts the number of independent paths through a method
    - Every decision point adds 1 to the complexity
    - A method with complexity 15 requires at minimum 15 test cases to achieve full branch coverage
    - In practice it rarely gets fully tested, which means it harbours untested paths that become production bugs
  - Using `max=10` on forces developers to decompose complex methods into smaller, single-responsibility methods

```java
// Complexity: 1 (no branches)
public String getFullName() {
    return firstName + " " + lastName;
}

// Complexity: 4 (1 base + 3 decision points)
public BigDecimal calculateBonus(Employee employee) {
    BigDecimal base = employee.getSalary().multiply(new BigDecimal("0.1"));

    if (employee.getYearsOfService() > 10) {      // +1
        base = base.multiply(new BigDecimal("1.5"));
    }

    if (employee.getDepartment() == Department.SALES) {  // +1
        base = base.multiply(new BigDecimal("1.2"));
    }

    return employee.isManager()                   // +1
        ? base.multiply(new BigDecimal("2.0"))
        : base;
}
```

Cyclo	Meaning	Action
1-5	Simple, easy to test	Fine
6-10	Moderate – acceptable with good tests	Monitor
11-15	Complex – hard to test completely	Refactor soon
16+	Very complex – almost certainly a design problem	Refactor now

Checkstyle

- The suppressions file tells Checkstyle to ignore specific rule violations in specific places
  - Prevents the ruleset from becoming so rigid that it blocks legitimate work
  - Without a suppressions file, every rule in your checkstyle.xml applies to every Java file in your project with no exceptions
  - There exist situations where a rule violation is either unavoidable, irrelevant, or would require more harm than good to fix

```
<?xml version="1.0"?>
<!DOCTYPE suppressions PUBLIC
    "-//Checkstyle//DTD SuppressionFilter Configuration 1.0//EN"
    "https://checkstyle.org/dtds/suppressions_1_0.dtd">

<suppressions>

    <suppress
        files="pattern that matches file paths"
        checks="pattern that matches rule names"
        lines="specific line numbers (optional)"
        columns="specific column numbers (optional)"
        message="pattern that matches the violation message (optional)"
    />

</suppressions>
```

Checkstyle

- Where Checkstyle should not apply
  - Generated code
    - JPA metamodel classes, Protocol Buffer generated classes, JOOQ database classes, and similar auto-generated sources
    - Frequently violates naming conventions, line length limits, and complexity rules
    - Not written by developers and should not be reviewed as if they were
  - Legacy code being migrated
    - Applying to an existing codebase, would produce thousands of violations that have nothing to do with the work in progress
    - A pragmatic migration strategy suppresses violations on legacy files while enforcing rules strictly on new code

```
<?xml version="1.0"?>
<!DOCTYPE suppressions PUBLIC
    "-//Checkstyle//DTD SuppressionFilter Configuration 1.0//EN"
    "https://checkstyle.org/dtds/suppressions_1_0.dtd">

<suppressions>

    <suppress
        files="pattern that matches file paths"
        checks="pattern that matches rule names"
        lines="specific line numbers (optional)"
        columns="specific column numbers (optional)"
        message="pattern that matches the violation message (optional)"
    />

</suppressions>
```

Checkstyle

- Where Checkstyle should not apply
  - Test code
    - Legitimately use magic numbers as test data
    - Have long method bodies that set up complex scenarios
    - Use naming conventions that violate production naming rules
  - Framework constraints
    - Some frameworks require specific patterns that violate style rules
    - For example, Spring's @Configuration classes sometimes have long methods
    - JPA entities sometimes require default constructors that Checkstyle's design rules would flags

```
<?xml version="1.0"?>
<!DOCTYPE suppressions PUBLIC
    "-//Checkstyle//DTD SuppressionFilter Configuration 1.0//EN"
    "https://checkstyle.org/dtds/suppressions_1_0.dtd">

<suppressions>

    <suppress
        files="pattern that matches file paths"
        checks="pattern that matches rule names"
        lines="specific line numbers (optional)"
        columns="specific column numbers (optional)"
        message="pattern that matches the violation message (optional)"
    />

</suppressions>
```

Suppression in Practice

- Applying suppression rules
  - Can specify which files, directories and rules should be suppressed
  - Regex can be used for both arguments
  - The suppression file is referenced in the pom.xml alongside the main rules file

```
<!-- Suppress in a specific directory -->
<suppress files="generated-sources" checks=".*"/>

<!-- Suppress for all test files -->
<suppress files=".*Test\.java" checks="MagicNumber"/>

<!-- Suppress for a specific file -->
<suppress files="EmployeeMapper\.java" checks="CyclomaticComplexity"/>

<!-- Suppress for all files in a specific package path -->
<suppress files="com[\\/]bank[\\/]legacy[\\/]" checks=".*"/>

<!-- Note: [\\/] matches both forward slash (Linux/Mac)
     and backslash (Windows) in file paths -->
```

```
project-root/
├── config/
|   └── checkstyle/
|       ├── checkstyle.xml
|       └── suppressions.xml
├── pom.xml
└── src/
```

Suppressions in Practice

- Applying suppression rules
  - The de facto standard location in enterprise Java projects is config/checkstyle/
  - Suppressions use regex patterns to match file paths and rule names
  - The [\\/] pattern matches both forward slash (Linux/macOS) and backslash (Windows) in file paths
    - Essential for cross-platform teams

```
<!-- Suppress in a specific directory -->
<suppress files="generated-sources" checks=".*"/>

<!-- Suppress for all test files -->
<suppress files=".*Test\.java" checks="MagicNumber"/>

<!-- Suppress for a specific file -->
<suppress files="EmployeeMapper\.java" checks="CyclomaticComplexity"/>

<!-- Suppress for all files in a specific package path -->
<suppress files="com[\\/]bank[\\/]legacy[\\/]" checks=".*"/>

<!-- Note: [\\/] matches both forward slash (Linux/Mac)
     and backslash (Windows) in file paths -->
```

```
project-root/
├── config/
│   └── checkstyle/
│       ├── checkstyle.xml
│       └── suppressions.xml
├── pom.xml
└── src/
```

Code Inspections

Code Inspections

- IntelliJ's static analysis engine runs continuously as you edit
  - Inspections operate on IntelliJ's fully resolved semantic model
  - This is not textual token parse
  - Three severity levels surface in the editor
    - Shown in the sidebar
- Inspection results appear in
    - The editor gutter, the coloured stripe on the right margin
    - The Problems tool window (Alt+6)
    - The Analyze → Inspect Code full-project report

Severity	Colour	Meaning
Error	Red underline	Definite problem — will not compile or will fail at runtime
Warning	Yellow underline	Probable problem — code is valid but likely wrong
Weak Warning	Grey underline	Improvement available — correct but suboptimal

Inspections vs SAST

- IntelliJ inspections
  - Serve different purposes and catch different problems than enterprise SAST tools, such as Checkmarx
  - These tools are complementary, not alternatives
    - IntelliJ inspections are not a substitute for SAST
    - SAST does not replace the value of catching quality issues at edit time
  - SAST tools perform taint analysis
    - They track how user-controlled input flows through the application
    - Determine whether it reaches a dangerous sink (SQL query, an HTML output, a file path) without sanitization.

	IntelliJ Inspections	SAST (e.g. Checkmarx)
When it runs	Continuously, as you type	CI/CD pipeline, on commit or PR
Scope	Current file and resolved type graph	Entire application, cross-service
Primary focus	Code quality, correctness, Spring config	Security vulnerabilities, taint analysis
Analysis depth	Semantic model of local codebase	Data flow across application boundaries
Example catches	Null dereference, unused bean, `==` vs `.equals()`	SQL injection, XSS, SSRF, insecure deserialization
Feedback speed	Immediate — milliseconds	Minutes to hours
Audience	Developer	Developer + Security team

Inspections vs SAST

- IntelliJ inspections
  - Catches the kind of problems that make code wrong or unmaintainable
    - Null pointer risks, resource leaks, misconfigured Spring components
  - Professional standards, before any code merges
    - Zero unacknowledged IntelliJ errors
    - Clean SAST results

	IntelliJ Inspections	SAST (e.g. Checkmarx)
When it runs	Continuously, as you type	CI/CD pipeline, on commit or PR
Scope	Current file and resolved type graph	Entire application, cross-service
Primary focus	Code quality, correctness, Spring config	Security vulnerabilities, taint analysis
Analysis depth	Semantic model of local codebase	Data flow across application boundaries
Example catches	Null dereference, unused bean, `==` vs `.equals()`	SQL injection, XSS, SSRF, insecure deserialization
Feedback speed	Immediate — milliseconds	Minutes to hours
Audience	Developer	Developer + Security team

Alt+Enter Workflow

- Alt+Enter is an important productivity key in IntelliJ
  - Opens a context-sensitive menu of available fixes, intentions, and transformations
  - The options change completely depending on what is under the cursor
  - Similar to ctrl+. in Visual Studio or VS Code
    - Has significantly more depth
    - Because IntelliJ's intention actions are informed by the full semantic model

Cursor position	Available actions
Red error underline	Quick fixes — correct the problem
Yellow warning	Suggested improvements
Any code element	Intentions — valid transformations that may change behaviour
Missing import	Add import, or show all candidates if ambiguous
`@Autowired` field	Convert to constructor injection
Lambda expression	Convert to method reference, or expand to anonymous class
`if` statement	Invert condition, merge with outer `if`, convert to ternary
String concatenation in loop	Convert to `StringBuilder`

Spring-Specific Inspections

- IntelliJ Ultimate includes a Spring plugin that resolves the Spring application context at edit time
  - These inspections catch configuration mistakes before the application starts
  - Each of these inspections represents a class of runtime failure that previously required starting the application to discover
  - More on this after the Spring Boot module

Inspection	What It Catches	Why It Matters
Unresolved bean dependency	`@Autowired` parameter has no matching bean in context	Spring throws `NoSuchBeanDefinitionException` at startup — caught at edit time
Circular dependency	Two or more beans depend on each other, directly or indirectly	Spring Boot 2.6+ fails on circular dependencies by default
`@Transactional` **on non-public method**	Transaction annotation on `private` or package-private method	Spring's proxy-based AOP cannot intercept these — the annotation is silently ignored
Unresolved `@Value` **expression**	`@Value("${property.key}")` references a key not in any loaded config	Causes `IllegalArgumentException` at startup with a confusing message
`@RequestMapping` **URL collision**	Two endpoints mapped to the same HTTP method + path	Spring throws `BeanDefinitionStoreException` at startup

Refactoring

Refactoring

- Refactoring is the process of restructuring existing code without changing its observable behaviour
  - The code does the same thing before and after
  - What changes is the internal structure: how it is organized, named, and decomposed
- The formal definition
  - Martin Fowler, Refactoring, 1999
  - A disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behaviour
- Refactoring is not rewriting, not fixing bugs, and not adding features
  - It is a precise, incremental improvement to the design of working code

Refactoring

- Why code needs refactoring
  - Code quality degrades naturally over time, referred to as technical debt
  - Every shortcut taken under time pressure, every quick fix, every feature added without updating the design accumulates
  - Code that started clean becomes progressively harder to understand, modify, and test
- Four symptoms (code smells) that signal refactoring is needed
  - Duplication: the same logic appears in multiple places. A bug fixed in one place reappears in another
  - Long methods: a method that does too many things is hard to name, hard to test, and hard to change safely
  - Poor naming: a class called Manager, a variable called data, a method called process() communicate nothing about intent
  - Tight coupling: components that know too much about each other cannot be changed or tested independently

The Accumulation Problem

- Code is written once but read and modified many times
  - Each modification to poorly structured code is slower, riskier, and more likely to introduce new bugs
  - The cost is not linear but compounds
  - The code that most needs refactoring is the code that is most dangerous to refactor because it has no tests
  - Refactoring is best done continuously, in small steps, before debt accumulates

```
Initial state: clear, simple, well-named
       ↓ Feature added quickly, no time to restructure
       ↓ Bug fixed with a conditional in an existing method
       ↓ Another feature — easier to copy-paste than to abstract
       ↓ Method now handles five different cases
       ↓ Nobody is sure what it does without reading all of it
       ↓ Tests are hard to write because there are too many paths
       ↓ Every change risks breaking something

Final state: the method nobody wants to touch
```

Refactoring and Testing

- Refactoring without tests is risky
  - Tests are the mechanism that verifies behaviour is preserved after each transformation
  - The refactoring discipline and the testing discipline are inseparable

1. Ensure the code has tests that cover the behaviour you are about to change
2. Make one small refactoring
3. Run the tests
4. Green → commit or continue
5. Red → undo the refactoring and understand why
6. Repeat

Refactoring vs. Other Activities

- The discipline of keeping these activities separate is what makes each one safe
  - Do not add features while refactoring because you will not know which change introduced a problem
  - Do not refactor while fixing a bug because you will not know which change fixed it

Activity	Changes Behaviour?	Changes Structure?	When to Do It
Refactoring	No	Yes	Continuously, before and after feature work
Bug fixing	Yes (corrects it)	Minimally	When a defect is identified
Feature development	Yes (adds it)	Yes	When a requirement exists
Performance optimisation	Sometimes	Yes	When a measured bottleneck exists
Rewriting	Potentially	Yes	Rarely — when structure is irreparable

Vocabulary of Refactoring

- Decomposition refactorings: breaking things apart
  - Extract Method: pull a block of code into a named method
  - Extract Class: pull a set of related fields and methods into a new class
  - Extract Interface:define an abstraction from a class's public methods
- Consolidation refactorings:bringing things together
  - Inline Method: replace a method call with its body when the method adds no clarity
  - Collapse Hierarchy: merge a subclass into its superclass when the distinction is no longer meaningful

Vocabulary of Refactoring

- Naming refactorings: making intent explicit
  - Rename: rename any symbol to better express what it represents
  - Introduce Explaining Variable: assign a complex expression to a named variable
- Structural refactorings: reorganizing relationships
  - Move Method / Move Class: relocate code to the class or package where it belongs
  - Introduce Parameter Object: replace a long parameter list with a dedicated object
  - Replace Conditional with Polymorphism: replace if/else or switch on type with subclass dispatch

Refactoring Example

- Extract Method is a commonly applied refactoring in practice
  - Its effect is immediate
  - Long methods become shorter
  - Intent becomes explicit
  - Testability improves
  - Coupling is loosened
- The before code is in the sidebar

```
public void processOrder(Order order) {
    // Validate the order
    if (order.getItems().isEmpty()) {
        throw new IllegalArgumentException("Order has no items");
    }
    if (order.getCustomer() == null) {
        throw new IllegalArgumentException("Order has no customer");
    }

    // Calculate the total
    BigDecimal total = BigDecimal.ZERO;
    for (OrderItem item : order.getItems()) {
        total = total.add(item.getPrice().multiply(
            BigDecimal.valueOf(item.getQuantity())));
    }

    // Apply discount
    if (order.getCustomer().isPremium()) {
        total = total.multiply(new BigDecimal("0.9"));
    }

    order.setTotal(total);
}
```

Refactoring Example

- The code after the refactoring is shown in the sidebar
  - processOrder now reads as a description of what it does, not a description of how
  - Each extracted method is independently testable
  - Changing the discount calculation touches one method, not a buried block inside a larger one
  - Conforms better to the single responsibility principle

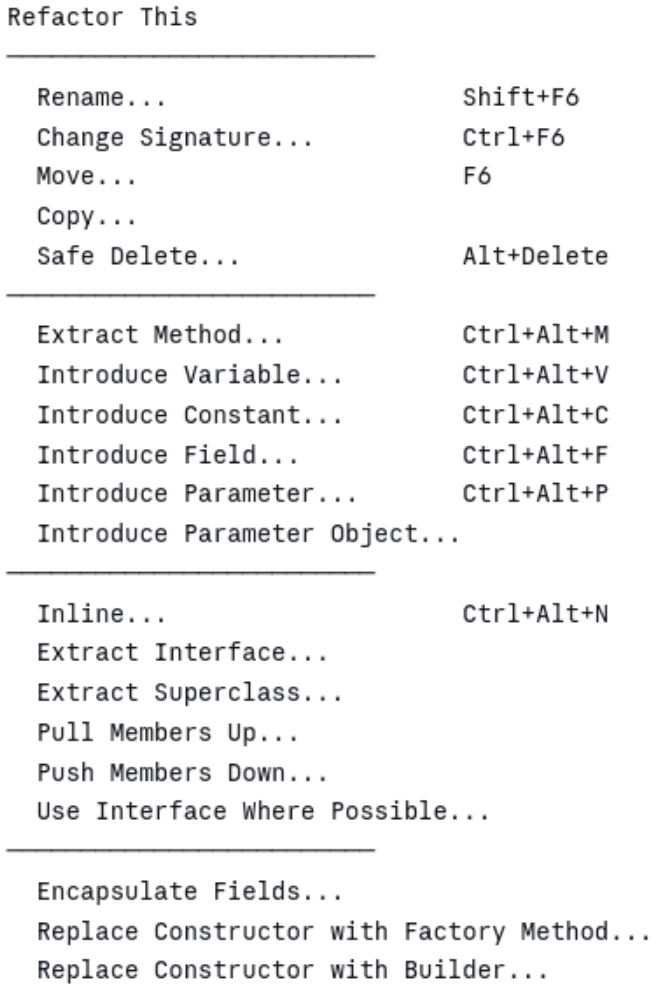
```
public void processOrder(Order order) {
    validateOrder(order);
    order.setTotal(calculateTotal(order));
}

private void validateOrder(Order order) {
    if (order.getItems().isEmpty()) {
        throw new IllegalArgumentException("Order has no items");
    }
    if (order.getCustomer() == null) {
        throw new IllegalArgumentException("Order has no customer");
    }
}

private BigDecimal calculateTotal(Order order) {
    BigDecimal subtotal = sumItemPrices(order.getItems());
    return applyCustomerDiscount(subtotal, order.getCustomer());
}
```

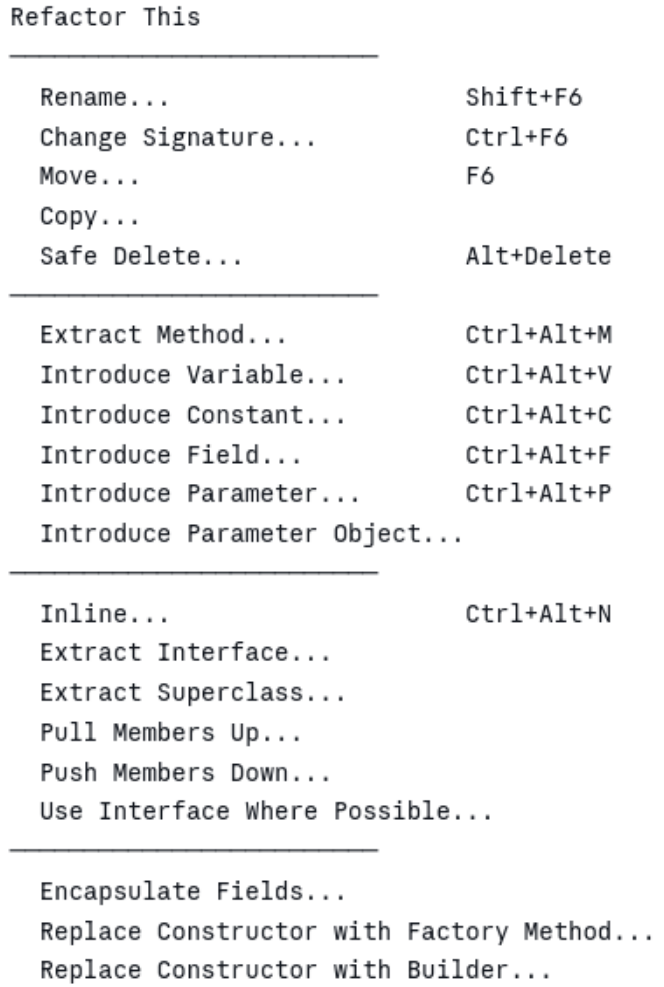
Refactoring in IntelliJ

- Safe refactoring in IntelliJ
  - IntelliJ's refactoring operations are semantic, not textual
  - Every operation works against the resolved type graph
    - All usages across all modules are found and updated
  - The result is guaranteed to compile after the refactoring



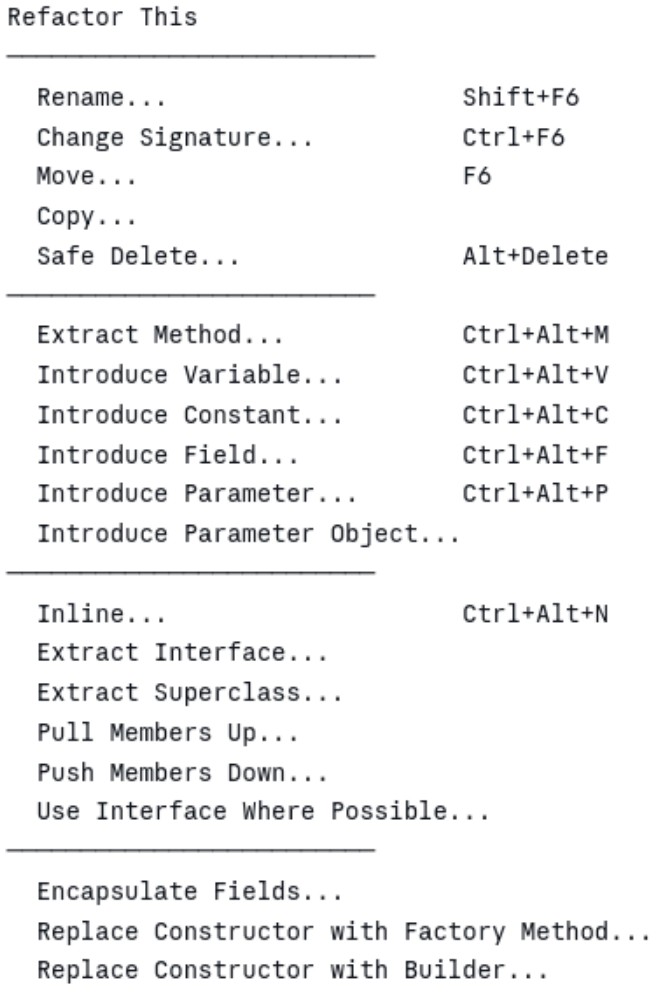
Refactoring

- Suppport for Java
  - Java's static typing means IntelliJ can trace every usage of a class, method, or field with certainty
  - A rename touches every call site, every import, every Javadoc reference, and every Spring XML config that references the class by name
  - A text search-and-replace cannot do this
    - It is not aware of scope, overloading, or shadowing



Refactoring

- Support for Java
  - The guarantee that a refactoring produces code that compiles is important
  - In a large codebase with hundreds of classes, a method rename touches dozens of call sites across different packages and modules
  - IntelliJ's refactoring engine resolves the exact set of references to the specific symbol being renamed
    - Excluding different symbols that happen to share the same name
    - Method overloading, shadowed variables, same-named methods in unrelated classes



Refactoring

IntelliJ Refactoring	Shortcut	Use Case
Rename	`Shift+F6`	Rename class, method, field, variable, or parameter across all usages, including imports, Javadoc, and Spring config
Extract Method	`Ctrl+Alt+M`	Pull a selected block of code into a new named method. IntelliJ infers parameters and return type
Extract Variable	`Ctrl+Alt+V`	Assign an inline expression to a named local variable
Extract Constant	`Ctrl+Alt+C`	Promote a magic literal to a named `static final` constant
Extract Field	`Ctrl+Alt+F`	Promote a local variable to a class field
Extract Parameter	`Ctrl+Alt+P`	Promote a hardcoded value to a method parameter
Inline	`Ctrl+Alt+N`	Inline a method or variable — remove a layer of indirection
Move Class	`F6`	Move a class to a different package. Updates all imports across the project
Change Signature	`Ctrl+F6`	Add, remove, reorder, or rename method parameters. Updates all call sites
Introduce Parameter Object	`Ctrl+Alt+Shift+T`	Replace a long parameter list with a dedicated class or record

Codebase Navigation

Large Codebases

- The navigation problem
  - A large Java codebase is not a collection of files, it is a graph of relationships and logical dependencies
    - Classes depend on interfaces
    - Interfaces have multiple implementations
    - Spring beans wire together at runtime
  - The challenge is not finding a file but understanding how the different artifacts are connected
- What makes large Java codebases hard to navigate
  - Hundreds or thousands of classes across dozens of packages
  - Deep inheritance hierarchies and interface chains
  - Spring dependency injection wires components together invisibly at runtime
    - Means you cannot follow the code directly to understand the relationships
  - Business logic distributed across layers: controllers, services, repositories, domain objects
  - Code written by many developers over many years with inconsistent naming

Large Codebases

- The wrong approach
  - Browsing the file tree looking for the right class
  - Searching for text strings and hoping the results are manageable
  - Asking a colleague every time you need to find something
- The right approach
  - Using the IDE's semantic navigation to follow the type graph, not the file system
  - A file tree is a physical organization that tells you where code is stored
  - A type graph is the logical organization that tells you how code is structured
- Every navigation feature discussed in this section operates on the resolved type graph, not on file names or text content

Three Navigation Modes

- IntelliJ provides three fundamentally different navigation modes
  - Each suited to answering a different type of question
- Name-based navigation
  - "I know what I'm looking for"
  - You know the name of the class, method, or file
  - Use: Ctrl+N (class), Ctrl+Shift+N (file), Shift+Shift (anything)
- Structure-based navigation
  - "I know where I am, I want to explore"
  - You are in a class and want to understand its shape or find a specific member
  - Use: File Structure popup Ctrl+F12, Project tool window Alt+1

Three Navigation Modes

- Relationship-based navigation
  - "I want to understand how this connects"
  - You have a symbol and want to understand its relationships with the rest of the system
  - Use: Find Usages Alt+F7, Go to Declaration Ctrl+B, Go to Implementation Ctrl+Alt+B, Hierarchy views Ctrl+H / Ctrl+Alt+H
  - Most developers in an unfamiliar codebase need relationship-based navigation
- This is where IntelliJ's semantic model provides the most value

Name-Based Navigation

- Name-based navigation
  - Efficient only when you already know what you are looking for
  - Which means you probably already have some familiarity with the codebase
- Go to class: Ctrl+N
  - Opens a search dialog for any Java class in the project or its dependencies
  - Supports camelCase abbreviation: typing OrdSvc finds OrderService
  - Supports wildcard: Order* finds all classes beginning with Order
  - Add #methodName to navigate directly to a method: OrderService#calculateTotal

Name-Based Navigation

- Go to file: Ctrl+Shift+N
  - Finds any file by name, regardless of type: .java, .xml, .yml, .properties
  - Essential for navigating to application.properties, pom.xml, or any non-Java resource
- Search everywhere: Shift+Shift (double-tap Shift)
  - Searches across classes, files, symbols, actions, and IDE settings simultaneously
  - Tabs filter results: All / Classes / Files / Symbols / Actions / Git
  - The Actions tab finds IDE menu items by name useful when you cannot remember where a setting lives
- Go to symbol: Ctrl+Alt+Shift+N
  - Searches across all named symbols: methods, fields, constants, across all classes

Structure-Based Navigation

- Structure-based navigation
  - Understanding the shape of a single class without scrolling through it
- File structure popup: Ctrl+F12
  - Shows all members of the current class in a popup: fields, constructors, methods, inner classes
  - Type to filter: in a class with 40 methods, type find to immediately see all methods containing "find"
  - Press Enter to jump to the selected member
  - Shows inherited members when toggled
    - See everything available on the class, not just what is declared in it

Structure-Based Navigation

- The project tool window: Alt+1
  - The file tree on the left side of the IDE
  - Alt+F1 → Select in Project: locates the currently open file in the tree
  - Useful for understanding where a class sits relative to its package siblings
  - Not the primary navigation tool
  - Use it for orientation, not exploration
- Navigate between members: Alt+Down / Alt+Up
  - Jump to the next or previous method in the current file
  - Faster than scrolling when reviewing a class top to bottom

Structure-Based Navigation

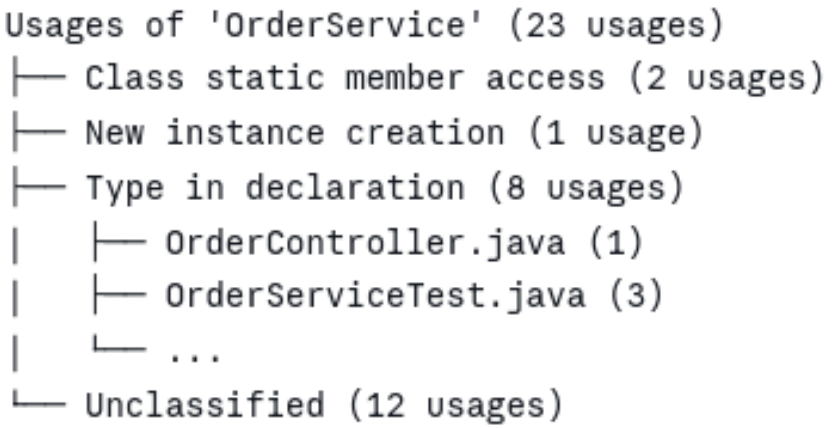
- Recent files: Ctrl+E
  - Shows the files you have visited recently, in order
  - Keeps you oriented across a session: navigation history at a glance
- Recent locations: Ctrl+Shift+E
  - Shows recently edited or viewed code locations, with a snippet of context
  - More granular than Recent Files because it shows which part of a file you were in

Structure-Based Navigation

- Go to declaration: Ctrl+B
  - Jumps to where a class, method, field, or variable is defined
  - On a method call: jumps to the method body
  - On a type: jumps to the class declaration
  - On an import: jumps to the class being imported
  - On a Spring @Value("${property.key}"): jumps to the property definition in the YAML/properties file
- Go to implementation: Ctrl+Alt+B
  - Jumps to the concrete implementation(s) of an interface method or abstract method
  - If one implementation exists: jumps directly
  - If multiple implementations exist: shows a popup listing all of them

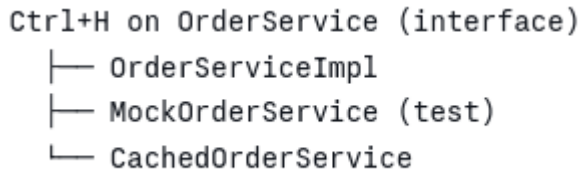
Structure-Based Navigation

- Find usages: understanding impact
  - Alt+F7 on any symbol answers the question "what depends on this?"
  - Essential before any change, any refactoring, any deletion
  - Shows
    - Every call site for a method, across all files and modules
    - Every field access (read and write shown separately)
    - Every class instantiation (new OrderService(...))
    - Every type reference (parameter type, return type, field type, generic argument)
    - Usages in test code, shown separately from production code
    - Usages in Spring XML configuration and annotation-based configuration



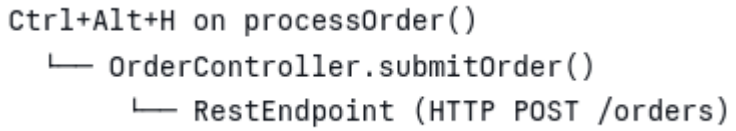
Hierarchy Navigation

- Hierarchy navigation
  - Views answer structural questions about the type system
  - Essential for understanding inheritance and call chains in an unfamiliar codebase
- Type hierarchy: Ctrl+H
  - Shows the full inheritance tree for any class or interface
  - Upward: all superclasses and interfaces this type implements
  - Downward: all subclasses and implementing classes



Hierarchy Navigation

- Method hierarchy: Ctrl+Shift+H
  - Shows all declarations of a method across the inheritance hierarchy
  - Which superclass defines it, which subclasses override it
- Call hierarchy: Ctrl+Alt+H
  - Shows all methods that call the selected method (callers)
  - And all methods those callers call (the full call chain)
  - Answers: "how does execution reach this code?"



Questions?